

A PROTOCOL WHITEPAPER — VERSION 2

SALVAGE

Sustainable liquidity and MEV defense on Uniswap v4, built on two real, still-enforceable bodies of maritime law: general average and the law of salvage.

"No cure, no pay."

"Salvage is the compensation allowed to persons by whose assistance a ship or her cargo has been saved, in whole or in part, from impending peril."

THE LAW OF SALVAGE — TRACEABLE TO THE EDICTS OF RHODES AND JUSTINIAN'S DIGEST

Table of Contents

- I. Executive Summary
- II. The Problem
- III. The Two-Law Engine
- IV. Architecture Overview (*Diagram 1*)
- V. The Manifest & the Loss Meter, in Detail
- VI. The Salvage Auction, in Detail
- VII. The Convoy Split, in Detail
- VIII. The General Average Fund, in Detail
- IX. Upcoming Features (*Diagram 2*)
- X. Positioning Against Prior Art
- XI. Build Roadmap
- XII. Honest Risks & Open Questions
- XIII. Glossary
- Sources Consulted

I. Executive Summary

A Uniswap pool is a price quote that's always a beat behind the real market. SALVAGE turns the correction of that lag into a fair, measured, self-funding transaction — instead of a silent tax on liquidity providers.

Three problems compound inside every concentrated-liquidity AMM: **loss-versus-rebalancing (LVR)**, the constant background leak as arbitrageurs trade against stale prices; **sandwich attacks**, where a bot trades immediately before and after a specific victim; and **JIT liquidity sniping**, where risk-free capital parachutes in for one fee and leaves. Most existing designs either tax or ignore the bots causing this. SALVAGE takes a third posture, borrowed from two real, still-binding bodies of maritime law — **general average** (loss-sharing) and the **law of salvage** (rewarding the corrector) — and implements both as a coordinated Uniswap v4 hook stack.

WHAT'S BUILT

The manifest tracks per-LP exposure. **The loss meter** measures realized LVR using Uniswap's own fee-accounting pattern, repointed at loss. **The salvage auction** makes bots bid for correction rights, with "no cure, no pay." **The convoy split** segments retail flow into sandwich-proof batches. **The general average fund** settles proportional claims automatically, every block.

WHAT'S NEXT

The Fleet Auction bundles correlated pools into one bid, splitting proceeds fairly by each pool's measured contribution. **The protected submission lane** closes a real gap in the convoy design by keeping retail orders out of the public mempool entirely, via private RPC submission.

Every historical and technical claim in this document has been checked against real, named prior art — CoW AMM, McAMM, Angstrom, Bunni, Bancor, BarnBridge — so the differentiation claims made here are specific and defensible, not asserted.

II. The Problem

A Uniswap pool only knows one price: whatever its last trade left it at. It has no way to observe the outside world between trades. The instant real-world price drifts away from that last-known price — which happens constantly — the pool is quoting *wrong*, and stays wrong until someone trades against it.

- **LVR (loss-versus-rebalancing)** — every time an arbitrageur trades against a stale quote, the value they capture comes directly out of LP capital. Milionis, Moallemi, Roughgarden and Zhang proved this loss has a closed form; it is the single largest component of value extracted from LPs, larger than sandwich attacks and front-running combined, costing LPs an estimated 5-7% of pool value annually even in high-volume pools.
- **Sandwich attacks** — a bot trades immediately before and after a specific victim's swap, extracting value from that trader directly by exploiting sequential, ordered execution.
- **JIT (just-in-time) liquidity sniping** — a bot deposits concentrated liquidity one block before a large trade, collects the fee, and withdraws immediately, holding risk for zero blocks.

WORKED EXAMPLE — HOW A \$10 GAP BECOMES REAL

Pool quotes ETH at \$2,990; the real market says \$3,000. A bot buys 1 ETH from the pool at \$2,990 and immediately resells it at \$3,000 — a guaranteed \$10 profit, extracted directly from whichever LP's capital was sitting in that price range. This single transaction is the entire mechanism LVR describes, replayed constantly, across every pool, every block.

Most existing designs pick one of two postures toward the bots causing this: **exile them** (taxes, delays, penalties) or **let them keep it**. SALVAGE takes a third, older posture.

III. The Two-Law Engine

Two distinct, real, still-binding doctrines of maritime law map almost exactly onto the two halves of this problem — rewarding the corrector, and compensating the sacrificed.

GENERAL AVERAGE

If a captain jettisons cargo to save the ship in a storm, the owner of that cargo doesn't bear the loss alone — every merchant whose goods survived owes a proportional contribution. Traceable to the Lex Rhodia, c. 800 BCE, preserved in Justinian's Digest 14.2.1, and still binding admiralty law today.

Maps to: LVR is the jettisoned cargo. LPs whose ranges got hit are compensated proportionally, funded by a shared pool.

LAW OF SALVAGE

Anyone who voluntarily rescues a ship or cargo in peril is owed a reward commensurate with the value saved — a "salvor." The reward only applies if the rescue actually succeeds: **no cure, no pay**. Formalized in the 1989 International Convention on Salvage.

Maps to: the arbitrage bot is the salvor — it corrects the pool's stale price, earning a reward sized to the value restored, via competitive bidding, only if its trade actually executes.

WHY BOTH LAWS, NOT ONE

General average alone answers "how do we compensate the victims" but says nothing about the party causing the correction. Salvage law alone rewards the corrector but has no mechanism for sharing the cost fairly among beneficiaries. Real maritime practice runs both simultaneously from the same casualty. SALVAGE runs them from the same event too: the salvor's reward (the auction bid) directly funds the general-average payout (the LP compensation), in one transaction.

IV. Architecture Overview

Every piece below is a hook or accounting layer living inside **one** v4 hook contract per pool — v4 permits exactly one hook address per pool, so the pieces below are internally modular but externally unified.

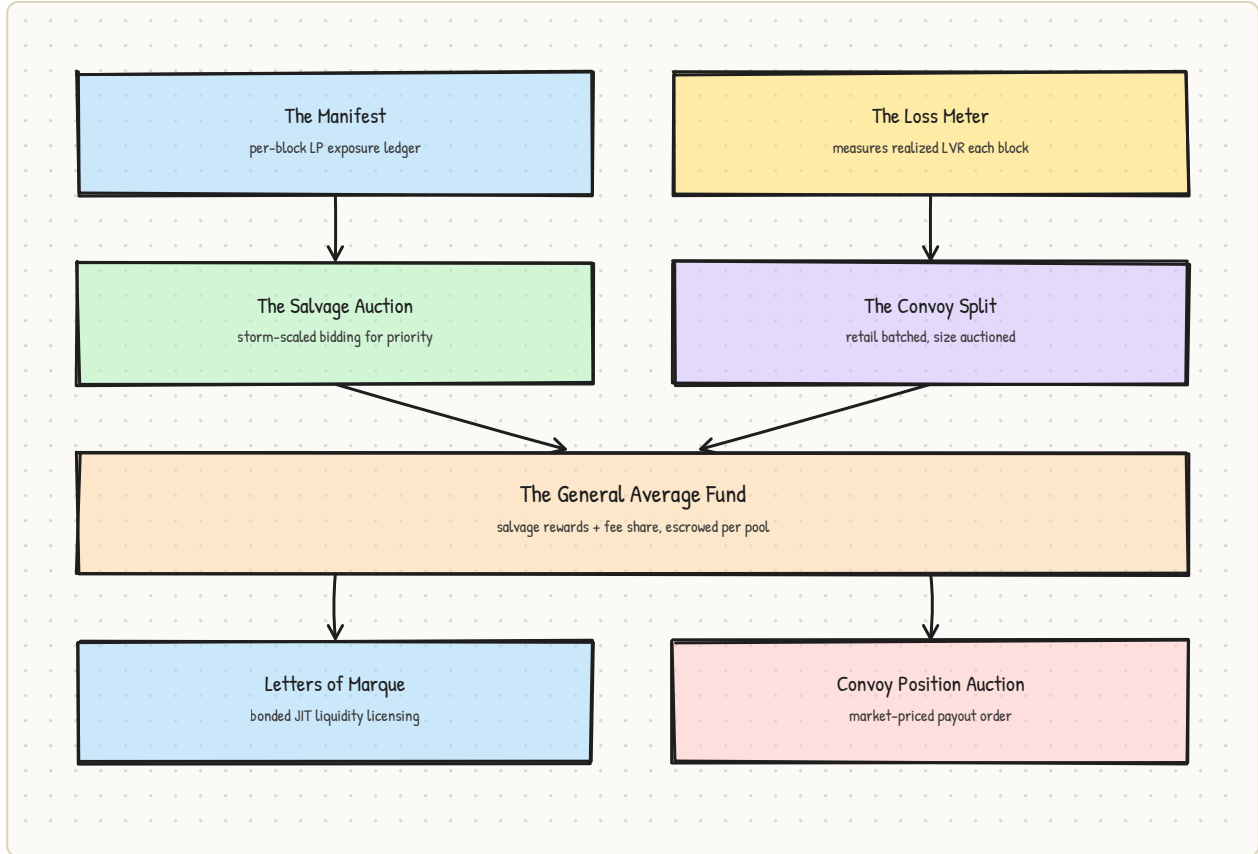


Diagram 1 — the core architecture. Yellow and blue measure; green and violet execute; orange settles; the bottom row extends fairness and liquidity licensing.

Sections V through VIII walk through the four core components in depth, each with a worked numeric example. Section IX introduces the two upcoming features and Diagram 2, showing exactly where they attach to this picture.

V. The Manifest & the Loss Meter, in Detail

The manifest — position-level exposure ledger

Snapshots every LP's tick-range exposure, piggybacking on v4's flash accounting. This is the precondition for measuring who was actually hit by anything — without it, no downstream payout can be fair.

The loss meter — reusing Uniswap's own accounting trick

Uniswap already tracks `feeGrowthGlobal` for every pool: total trading fees collected, divided by active liquidity at the time, accumulated into one running number. Every LP keeps a personal checkpoint of what that number read at their last interaction; their owed fees are simply $\text{liquidity} \times (\text{current} - \text{checkpoint})$.

WORKED EXAMPLE — FEEGROWTHGLOBAL

LP_A deposits 100,000 liquidity when the counter reads 0. A swap generates a \$300 fee while LP_A is the only liquidity active: counter becomes $300/100,000 = 0.003$. LP_B then deposits 50,000 liquidity, checkpointed at 0.003. A second swap generates \$450 with both LPs active (150,000 combined): counter becomes $0.003 + 450/150,000 = 0.006$. LP_A claims $100,000 \times (0.006 - 0) = \text{\$600}$. LP_B claims $50,000 \times (0.006 - 0.003) = \text{\$150}$. Total: \$750, matching both fees exactly — no LP overpaid, no LP shorted.

The loss meter reuses this exact structure — swap "loss" in wherever you'd see "fee." A pool-wide `lossGrowthGlobal` accumulator, per-tick snapshots at tick crossings (so a claim can be scoped to a specific LP's range, not just the whole pool), and one lazy checkpoint per LP position.

WORKED EXAMPLE — READING THE LOSS METER

Pool at \$2,990, true price \$3,000. A bot buys 10 ETH, walking the pool's execution price from \$2,990 up to \$3,000 as it fills. Average execution price $\approx \$2,995$. Bot pays the pool $10 \times \$2,995 = \$29,950$, resells at $10 \times \$3,000 = \$30,000$. Realized loss = **\$50** — this is the number the loss meter computes and feeds into the accumulator, attributing it tick-by-tick to whichever LP ranges the price path actually crossed.

HONEST ENGINEERING TRADE-OFF

Exact tick-by-tick integration is the mathematically correct approach but costs gas proportional to how many ticks a trade crosses. A cheaper simplification — one lump loss figure per swap, split by liquidity share across touched ticks — trades precision for cost. Both approaches need prototyping and gas-measurement before committing.

VI. The Salvage Auction, in Detail

An am-AMM / Angstrom-style priority auction, reframed as a salvage-reward market. Bots bid for the right to close a detected price gap; competition pushes the winning bid toward the gap's true value.

WORKED EXAMPLE — COMPETITIVE BIDDING ON A \$10 GAP

Bidder	Bid	Reasoning
Bot A	\$3	Opening bid — keep \$7 of the \$10
Bot B	\$5	A smaller profit beats zero
Bot A	\$7	Still profitable, worth defending
Bot B	\$8.80	Final winning bid

Bot B wins at \$8.80, which is **deposited into the general average fund before the trade executes**. Bot B then executes the actual arbitrage trade, earning the \$10 gross profit; its net take-home is $\$10 - \$8.80 = \$1.20$. The fund receives \$8.80 — the "salvage reward" — for the exposed LP.

NO CURE, NO PAY

The bid is only collected if the trade actually executes and closes the gap. If Bot B's transaction reverts — a gas spike, a race with another transaction — nothing is charged and nothing is paid. This prevents the fund from ever holding a payment for a rescue that didn't actually happen.

STORM-SCALED AUCTION WINDOW

Every existing delay/auction hook in this incubator's cohort uses a fixed bidding window, regardless of market conditions. A fixed short window (e.g. one block) starves competition during volatile periods — only the single fastest bot gets to bid, so the winning bid stays artificially low. SALVAGE widens the window under measured volatility (giving real competition time to form) and keeps it tight in calm markets (no unnecessary delay for ordinary traders).

HONEST TRADE-OFF

A longer window means ordinary users wait longer for finality during exactly the moments they're most anxious about price movement. This is a genuine tension, not a free win, and the exact curve relating window length to volatility needs careful tuning and testing.

VII. The Convoy Split, in Detail

Retail-sized swaps are grouped and cleared together at one uniform price — a frequent batch auction, per Budish, Cramton and Shim (2015) — so there is no ordering inside the batch to sandwich. Larger trades route through the salvage auction lane instead.

WHY BATCHING KILLS THE SANDWICH

A sandwich attack needs sequential execution: buy before the victim, sell after, profiting from the price movement the victim's own trade causes. Inside a uniform-price batch, every order — victim and bot alike — clears at the *same* price, regardless of arrival order. There is no "before" or "after" to exploit.

WHAT COUNTS AS "RETAIL"

Not a fixed dollar figure — the correct threshold is **price impact**, not raw size, since the same dollar amount can be trivial in a deep pool and disruptive in a thin one.

WORKED EXAMPLE — WHY PRICE IMPACT BEATS A FLAT DOLLAR CUTOFF

Trader	Trade size	Price impact	Classification
Retail buyer, deep pool	\$500,000	0.3%	Retail — convoy batch
Trader, thin altcoin pool	\$20,000	5%	Whale — salvage auction

A flat \$25,000 cutoff would misclassify both rows above. A price-impact threshold (e.g. 1%) correctly routes each trade based on what it actually does to the pool, not its face value.

VIII. The General Average Fund, in Detail

A per-pool escrow with **two distinct income streams**. Auction proceeds (the winning salvage bid) are funded and paid out in the same atomic transaction. The fee slice (a governance-set percentage of ordinary trading fees, diverted from LPs into the fund) genuinely accumulates as a standing balance over time, like any treasury buffer.

WORKED EXAMPLE — A SEVEN-BLOCK TIMELINE

Block	Event	Fund balance	lossGrowthGlobal
0	LP_X deposits 100,000 liquidity	\$0	0
2	Auction bid \$9.20 collected	\$14.20	0.000092
4	LP_Y deposits 50,000 liquidity	\$17.20	0.000092 (LP_Y's checkpoint)
5	Auction bid \$2.50 collected (150,000 active)	\$19.70	0.0001087

LP_X claims $100,000 \times (0.0001087 - 0) = \mathbf{\$10.87}$. LP_Y claims $50,000 \times (0.0001087 - 0.000092) = \mathbf{\$0.83}$. LP_X's figure correctly includes the full block-2 bid (LP_Y wasn't present yet) plus its proportional share of block 5; LP_Y only ever collects from block 5 onward.

THE BANCOR COMPARISON, PRECISELY

Bancor's 2021 IL-protection fund collapsed because claims were paid by minting new BNT on demand — disconnected from any specific loss event. Minting enough to cover a wave of simultaneous claims crashed BNT's own price. The structural difference here isn't "the fund never holds a balance" (the fee-slice stream genuinely does) — it's that every dollar in this fund, from either stream, was already earned from real trading activity in this exact pool. There is no mechanism here that could inflate its way out of a shortfall; a shortfall simply means claims wait or partially settle.

IX. Upcoming Features

Two additions, each closing a real gap rather than decorating the core design — pressure-tested the same way every other component in this document was.

THE FLEET AUCTION

UPCOMING

Sophisticated bots already routinely correct multiple correlated pools in a single transaction — standard existing arbitrage-routing practice. SALVAGE's core design only knows how to handle one pool, one fund, one bid at a time. The Fleet Auction lets a bot bundle corrections across pools into a single bid, split fairly across each pool's general average fund by a shared settlement layer sitting above the individual pool hooks.

WORKED EXAMPLE — SPLITTING A BUNDLED BID

ETH/USDC has a \$10 gap; ETH/USDT has a \$6 gap, same block, same underlying cause. A bot bundles both corrections and bids \$14 total instead of bidding separately (and paying gas twice). Each pool's own loss meter independently computes its own gap — \$10 and \$6 — and the settlement layer splits the bid by that ratio:

$$\text{ETH/USDC's share} = 14 \times (10 / 16) = \$8.75$$
$$\text{ETH/USDT's share} = 14 \times (6 / 16) = \$5.25$$

Neither pool's hook trusts the bot's claim about the split — each contributes its own independently-measured gap, and the ratio is mechanically derived, not proposed or negotiated.

Why this is substantive, not decorative: it makes the salvage auction more *competitive*, not just more convenient. A bot that amortizes one gas cost across two corrections can afford to bid more aggressively on each — more value flows back to LPs than if the pools were forced into separate, gas-inefficient auctions.

HONEST CAVEAT

The shared settlement layer is real, non-trivial new architecture — both pools' hooks must trust and correctly call into a common contract holding the bundled bid, atomically. This is exactly the kind of surface where a bug would be expensive, and needs the most careful testing of anything in this document.

PROTECTED SUBMISSION LANE

UPCOMING

The convoy split assumes retail trades safely reach the batch — but a trade submitted the normal way briefly sits in the public mempool before the batch resolves, which is exactly the visibility window a sandwich bot needs. This is a real, previously unstated gap in the convoy design, not a hypothetical one.

The fix borrows proven infrastructure rather than inventing new mechanism: Flashbots Protect and MEV Blocker already run private RPC endpoints that skip the public mempool entirely. Retail-sized trades submitted through this lane route directly to the batch solver, never touching a public mempool at all.

HONEST CAVEAT

This is the least novel of the two additions — mostly "use existing private-RPC infrastructure," not new design. But it is the most necessary: without it, the convoy split's core promise doesn't fully hold end-to-end.

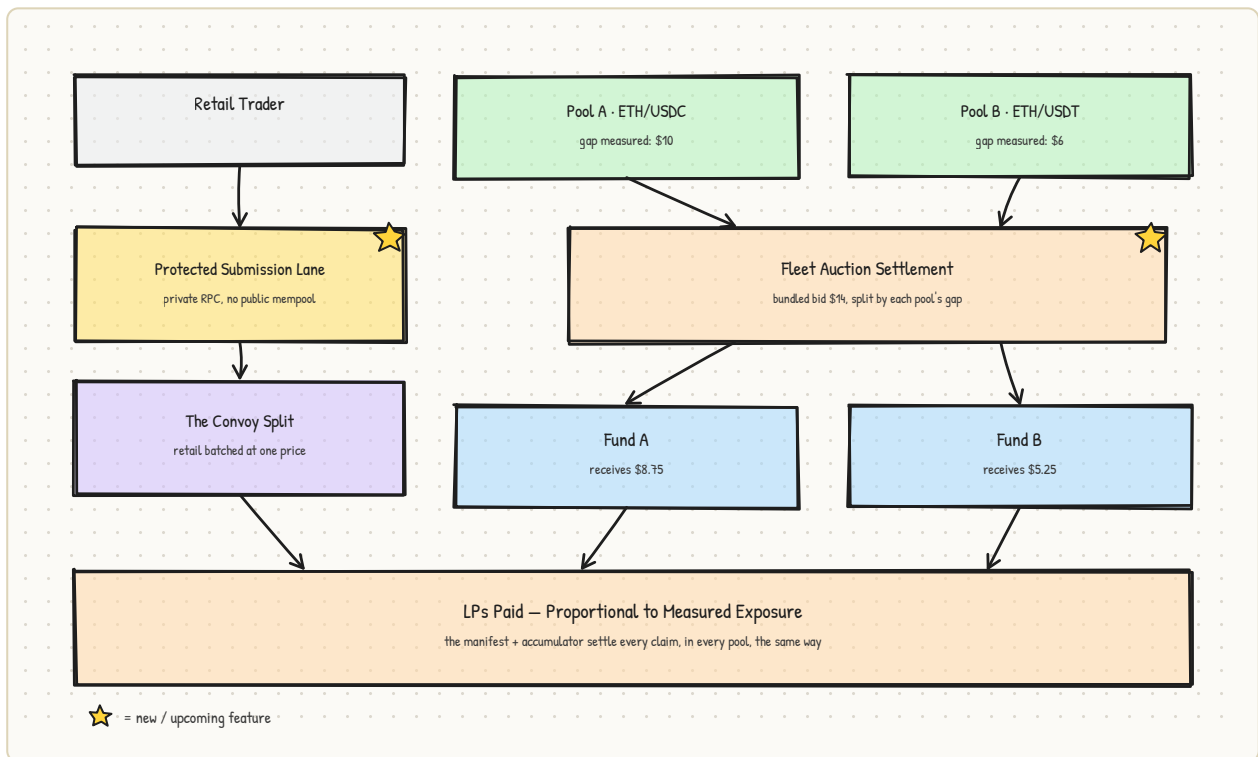


Diagram 2 — the updated architecture. Star-badged boxes are the two upcoming additions; the right branch replays the \$10 / \$6 / \$14 worked example exactly.

X. Positioning Against Prior Art

Prior art	What it does	The gap SALVAGE closes
CoW AMM (live, Balancer)	Batches trades; solver competition eliminates most LVR, surplus to LPs	Full-range pool, so an even split is already fair — doesn't need per-position accounting for v4's heterogeneous ranges
McAMM (2023 proposal)	Auctions the right to trade first, proceeds to LPs	Generic pro-rata proceeds — no per-position measured attribution
Angstrom / am-AMM / Bunni	Priority auctions and MEV recapture for v4 pools	Distribute roughly by liquidity depth, not measured per-block harm
Bancor IL Protection (2021, collapsed 2022)	LP insurance, claims paid from token issuance	Fund disconnected from the loss event — SALVAGE's fund is only ever filled by the exact cause of the loss
BarnBridge / Saffron / tranching-pool proposal	Junior/senior risk tranches for LPs	Fixed by governance vote — SALVAGE prices subordination continuously via auction

AGAINST THIS SPECIFIC INCUBATOR'S COHORT

Direction	Attempts	SALVAGE's position
Commit-reveal / delay ordering	18×	Deliberately avoided as the core mechanism — used only as a minor, storm-scaled parameter inside the salvage auction
CoW-style matching + fallback	17×	Not used — different mechanism family entirely
Batch-clearing settlement	~5×	Used, for the convoy split specifically, not as a standalone feature
Retail vs. whale segmentation	1×	Core differentiator — the convoy split
Volatility-scaled delay length	0-1×	Core differentiator — storm-scaled auction window
Cross-venue LVR recapture	thin	Byproduct — the salvage auction is venue-agnostic by construction

XI. Build Roadmap

1. **The manifest + loss meter** — the accumulator pattern, tested against simulated overlapping LP ranges. Publishable on its own before anything else exists.
2. **The general average fund**, fee-funded only at first — prove the proportional payout math and gas cost before adding the auction.
3. **The salvage auction**, fixed-window to start — bolt on "no cure, no pay" collection logic, route 100% of proceeds into the existing fund.
4. **Storm-scaled window** — the volatility-responsive auction length, your cleanest cohort differentiator to demo with a before/after chart.
5. **The convoy split** — retail batching; hardest core piece, since a true uniform-price batch needs an off-chain solver network. Budget the most time here.
6. **Protected submission lane** — closes the convoy split's mempool-visibility gap; mostly integration work against existing private-RPC infrastructure.
7. **Convoy position auction, letters of marque** — later-stage additions once the core is trustworthy.
8. **The Fleet Auction** — last, since it requires a shared settlement layer across multiple already-trustworthy pool hooks.

XII. Honest Risks & Open Questions

- LVR measurement depends on an oracle; a stale or wrong reference price corrupts every downstream payout. This is a real off-chain dependency.
- A true uniform-price convoy batch needs off-chain solvers to compute clearing prices, mirroring CoW Swap's architecture — the chain settles the result, but doesn't compute it.
- Six-plus interacting hooks is a large audit surface — each needs its own review plus an integration pass.
- The salvage auction only works with genuine bidder competition; a single uncontested bidder can win cheaply.
- The convoy position auction risks larger LPs consistently outbidding smaller ones for protected slots every epoch.
- The Fleet Auction's shared settlement layer is new, cross-pool architecture and the highest-risk surface in this document — bundling two funds' worth of money through one settlement point needs the most careful testing.
- Permissionless does not mean fully on-chain with zero external dependencies — worth stating precisely rather than overclaiming.
- This is a paper design. Every individual mechanism is proven elsewhere; this specific combination is untested.

XIII. Glossary

LVR (loss-versus-rebalancing)

The value an AMM loses to arbitrageurs trading against a stale price, compared to a hypothetical continuously-rebalanced portfolio. Has a proven closed form (Milionis et al.).

Salvor

In maritime law, a party who voluntarily rescues a ship or cargo in peril and earns a reward for doing so. Here: the bot that corrects a stale pool price.

General average

Maritime law principle requiring proportional loss-sharing among all who benefited from a sacrifice made for the common good.

No cure, no pay

Salvage-law principle: a reward is only owed if the rescue actually succeeds. Implemented here by only collecting an auction bid on successful trade execution.

feeGrowthGlobal / lossGrowthGlobal

A single running accumulator tracking fees (or losses) per unit of active liquidity, letting any number of LPs claim their share cheaply without looping.

Manifest

The ledger tracking each LP's tick-range exposure, block by block.

Convoy split

Segregating retail-sized trades into a uniform-price batch auction, separate from larger trades routed through the salvage auction.

Fleet Auction

Bundling salvage corrections across multiple correlated pools into one bid, split proportionally by each pool's independently measured gap.

Price impact

How much a given trade moves a pool's price — the correct basis for classifying a trade as "retail" or "whale," rather than raw dollar size.

— Sources Consulted

- Milionis, Moallemi, Roughgarden, Zhang — Automated Market Making and Loss-Versus-Rebalancing, arXiv:2208.06046
- Adams, Moallemi, Reynolds, Robinson — am-AMM: An Auction-Managed Automated Market Maker, FC 2025
- Budish, Cramton, Shim — The High-Frequency Trading Arms Race: Frequent Batch Auctions, QJE 130(4), 2015
- Law of salvage — edicts of Rhodes, Justinian's Digest, 1989 International Convention on Salvage, Lloyd's Open Form
- Lex Rhodia de lactu — Justinian's Digest 14.2.1; general average history
- CoW DAO — "The Risks of Providing Liquidity: Unmasking LVR in DeFi"; CoW AMM documentation
- DeFi Risk Transfer: Towards a Fully Decentralized Insurance Protocol, arXiv:2212.10308
- Quantstamp — "The Rise of Risk-Hedging DeFi Protocols" (BarnBridge, Saffron)
- Flashbots Protect; MEV Blocker — private RPC / mempool-protection documentation
- Atrium Academy Uniswap Hook Incubator — cohort submission analysis (commit-reveal, batch-clearing, segmentation, delay-length, cross-venue categories)